

SENG 465 – Artificial Intelligence in Game Programming

Group Members:

- Berfin Duru ALKAN — 202228005
- Şahin ERŞAN — 202128002
- Özgün SOYKÖK — 202228043
- İsmail DOĞAN — 202128045

Game Title:

"Intelligent Sudoku Solver and Analyzer"

Game Description:

This project presents a Sudoku-based game enhanced with artificial intelligence techniques. The player can attempt to solve Sudoku puzzles manually or request assistance from AI solvers. The system supports multiple difficulty levels and includes algorithm performance comparison features.

The core objective is to:

- Provide an interactive Sudoku gameplay environment.
- Apply constraint-based reasoning and search algorithms to solve puzzles.
- Compare three different solving algorithms under identical puzzle conditions.
- Allow difficulty scaling from simple 3×3 mini-Sudoku to full 9×9 puzzles.

Gameplay Mechanics:

Player Interaction:

- Players manually fill empty Sudoku cells.
- Optional real-time validation highlights mistakes.
- A hint system suggests valid values using the currently selected AI method.
- Players may request the puzzle to be solved automatically using one of the **three algorithms**:
 1. Constraint Propagation
 2. AC-3 (Arc Consistency Algorithm)
 3. Brute-force Backtracking

Difficulty Scaling:

- **Level 1:** Small 3×3 Sudoku, ideal for beginners.
- **Level 2:** Standard 9×9 Sudoku with moderate difficulty.
- **Level 3+:** Hard and expert puzzles with minimal clues and high constraint density. Difficulty progression increases the complexity of constraints and reduces available clues.

Winning Condition:

A puzzle is successfully completed when:

- Every row contains digits 1–N exactly once.
- Every column contains digits 1–N exactly once.
- Every sub-grid contains digits 1–N exactly once.
- No constraints are violated.

Losing Condition:

Sudoku does not traditionally include a fail state, but the system records:

- Attempts ending with an invalid board.
- Timeout failures (if time mode is enabled).
- User-abandoned puzzles.

Algorithms Used:

This project utilizes three algorithms:

1. Constraint Propagation
2. AC-3 (Arc Consistency Algorithm 3)
3. Brute-force Backtracking Search

How the Algorithms Work in the Game:

1. Constraint Propagation

Constraint Propagation progressively reduces domains through logical elimination:

1. Initialize all domains based on the puzzle.
2. Remove impossible values using Sudoku constraints.
3. When a cell becomes single-valued, propagate this information to its neighbors.
4. Continue until no further reductions are possible.
5. If unresolved, the simplified puzzle is passed to AC-3 or Backtracking.

2. AC-3 (Arc Consistency Algorithm)

AC-3 enforces binary consistency between related cells:

1. Represent puzzle as a CSP (variables, domains, constraints).
2. Place all arcs into a queue.
3. For each arc, revise domains to maintain consistency.
4. If a domain changes, add all related arcs back to the queue.
5. Continue until the system stabilizes or inconsistency occurs.

3. Brute-force Backtracking

Backtracking is a depth-first search solver:

1. Select an empty cell (optionally MRV heuristic).
2. Try possible values sequentially.
3. Check local consistency after each assignment.
4. Backtrack when contradictions arise.
5. Repeat until a full solution is found.

Experimental Comparison of the Three Algorithms:

The project compares the behavior and efficiency of the three solvers.

Metrics

- Runtime
- Number of visited nodes
- Backtracking count (for brute-force)
- Domain reduction count (for propagation algorithms)

Test Scenarios

- 3×3 Sudoku: Easy, Medium, Hard
- 9×9 Sudoku: Easy, Medium, Hard

Every puzzle is solved by all three algorithms.

Expected Observations

- Easy puzzles: All algorithms perform efficiently.
- Hard puzzles: Backtracking becomes slow with many backtracks.
- AC-3 and Constraint Propagation significantly reduce search space.
- Hybrid approaches (Propagation + AC-3 + limited Backtracking) show the best performance.

Levels and Difficulty:

- **Level 1:** Few clues, simplest constraints.
- **Level 2:** Larger grid with more complex placements.
- **Level 3+:** Expert difficulty with minimal clues, requiring advanced reasoning.

Additional Features:

Step-by-step Solver Visualization

- Shows domain reductions for Constraint Propagation and AC-3.

- Highlights trial assignments and backtracking steps for Backtracking.

Hint System

- Uses the selected algorithm to propose a valid value.

Score Tracking

- Tracks time, hints used, and algorithm-assisted solves.

Error Highlighting

- Incorrect moves highlight conflicting cells.

Summary:

This Sudoku project integrates constraint reasoning, CSP algorithms, and search methods to create a challenging and educational puzzle-solving environment. The inclusion of three distinct AI approaches enables a thorough comparison of performance and efficiency across varying puzzle complexities.

The system demonstrates:

- Practical AI integration in games.
- Effective use of Constraint Propagation and AC-3 for domain reduction.
- Performance contrast with brute-force search methods.

This final version aligns with the structure and detail level of the provided project concept while accurately representing your Sudoku-based AI system.